Retrieval-Augmented Generation over Jordanian Legislation

How the system is built, what every part does, and how data moves from a PDF on disk to a cited answer on screen.

Corpus	12 Jordanian laws and regulations (PDF and Word), 3,830 indexed passages, 1,069,453 tokens
Retrieval	Keyword (BM25) + meaning (bge-m3 embeddings), fused, then reordered by a cross-encoder - all on the local machine
Answers	Gemini 3.8 Flash or Claude Opus 5, grounded in retrieved articles, with citations back to the article
Code	4,149 lines of Python in 18 modules, 180 test functions, Python 3.12.10
Quality	The answering article is ranked first for 80% of a 49-question test set (43% before tuning); 94% within the top five

This document is generated by `docs/make_pdf.py` from a snapshot of the running system, so the counts and versions in it are read from the code and the index rather than written by hand.

Contents

1 What the system does . . . 3
2 Architecture at a glance . . . 3
3 Technology choices . . . 5
4 Stage 1 - Reading the documents . . . 6
5 Stage 2 - Repairing Arabic text . . . 7
6 Stage 3 - Finding the articles . . . 8
7 Stage 4 - Chunking and labelling . . . 9
8 Stage 5 - Embedding . . . 9
9 The index on disk . . . 10
10 Stage 6 - Retrieval . . . 10
11 Stage 7 - Reranking and the context budget . . . 11
12 Stage 8 - Generating the answer . . . 11
13 How data moves: every hop . . . 12
14 Interfaces: command line, HTTP, web page . . . 14
15 Configuration . . . 15
16 How the system was measured . . . 15
17 Testing . . . 16
18 Limitations and next steps . . . 17
Appendix A File-by-file map . . . 19
Appendix B Glossary . . . 20

1 What the system does

The system answers questions about a collection of documents by finding the relevant passages first and then asking a language model to answer using only those passages. That pattern is called retrieval-augmented generation, or RAG. The alternative - asking a model directly - produces fluent text that cannot be checked and may be invented. Here, every sentence of an answer points back to an article of a real law.

The corpus is Jordanian legislation: the Constitution, the Civil Code, the Penal Code, the Labour Law, the Companies Law, the Commercial Law, the Code of Criminal Procedure, the Execution Law, the Evidence Law, the Courts Formation Law, the Real Estate Ownership Law and the Compulsory Vehicle Insurance Regulation.

A question in Arabic or English goes in; an answer comes back in the language it was asked, citing the law and the article number, with the source passages listed underneath.

> **The one-paragraph version.** Each law is split into its articles. Every article is turned into a list of numbers that captures its meaning (an embedding) and into a bag of its words (a keyword index). A question is converted the same way, the two searches run in parallel, their rankings are merged, and a second model re-reads the best twenty passages to put the best one first. Roughly ten articles then go to Gemini with instructions to answer only from them. Everything except that last step runs on this machine.

Why not simply give the model the whole law?

The twelve documents hold about 1.07 million tokens. Sending them with every question would be slow, expensive, and worse: models answer less accurately when the relevant sentence is buried in a large context. Retrieval narrows 1.07 million tokens down to roughly 4,000 before the model sees anything - about 0.4% of the corpus, chosen for relevance.

2 Architecture at a glance

Two paths run through the system. Ingestion happens once per document and is expensive; querying happens per question and is fast. They meet at the index.

```
INGESTION  (runs once per document, on this machine)

documents/*.pdf, *.docx
         |
         v
[1] read            pypdf / zipfile+XML        raw text + page offsets
         |
         v
[2] repair Arabic   rag/arabic.py              readable, reordered text
         |
         v
[3] split articles  rag/legal.py               "Article 61" ... boundaries
         |
         v
[4] chunk + label   rag/chunking.py            one passage per article
         |
         +---------------------------+
         v                           v
[5] embed (bge-m3)            build keyword index
    1024 numbers each          (BM25, in memory)
         |                           |
         v                           v
     .rag_index/embeddings.npy   .rag_index/chunks.jsonl


QUERY  (runs per question)

question --> [6] two searches in parallel
                  |                    |
             keyword (BM25)       meaning (bge-m3)
                  |                    |
                  +---------+----------+
                            v
                  [7] merge rankings (RRF) --> drop near-duplicates (MMR)
                            v
                  [8] cross-encoder rescores top 20        ~2 s, local
                            v
                  [9] fill 4,000 tokens with best passages
                            v
                  [10] Gemini / Claude  <-- the only step that leaves the machine
                            v
                     answer + [n] citations
```

The numbered stages above are the sections of this document. Each is a separate module, and the arrows are plain Python function calls inside one process - there is no message queue, no database server and no network hop until stage 10.

The modules

Module	Lines	Responsibility
loaders.py	578	Read PDF, Word, text, CSV, JSON and HTML into one Document shape; cache extractions; skip duplicates
arabic.py	273	Normalise Arabic spelling for search; repair four distinct PDF extraction defects
legal.py	146	Find article boundaries in legislation, in three heading styles
chunking.py	356	Turn documents into passages: one article each, labelled, with offsets back to the source
embeddings.py	170	Run the embedding model locally (or Voyage AI if configured)
bm25.py	130	Keyword ranking, implemented directly so the tokenizer can be shared
store.py	367	The index: load, save, add, remove, deduplicate, detect stale settings
retriever.py	267	Hybrid search: fusion, diversification, filtering, reranking hand-off
crossencoder.py	100	Local reranker; int8 quantisation and length-sorted batching
llm.py	396	Gemini and Claude behind one interface; retries; error translation
generate.py	164	The prompt, the citation parsing, the refusal handling
pipeline.py	269	The facade the CLI, the API and your own code all call
cli.py	388	Terminal interface: ingest, search, ask, chat, stats, serve
server.py	216	HTTP API and the test page
config.py	207	Every setting in one dataclass; .env loading

Supporting files: `rag/web/index.html` (342 lines, the test page), `eval/evaluate.py` (347 lines, the measurement harness), and 180 test functions under `tests/`.

3 Technology choices

Every dependency here earns its place; the unusual choices are explained in the last column, and the measurements behind them are in section 16.

Layer	Choice	Version	Why this one
Language	Python	3.12.10	The whole ecosystem for text and models lives here
PDF reading	pypdf	6.18.1	Pure Python, no system dependency, and exposes both plain and layout extraction - the two modes turned out to be necessary to repair these files
Word reading	zipfile + ElementTree	stdlib	A .docx is a zip of XML; reading it directly avoids a dependency for ~60 lines of code
Neural models	PyTorch + Transformers	2.11.0 / 5.5.4	Runs the embedding and reranking models locally on the CPU
Embeddings	BAAI/bge-m3	1024 dimensions	Multilingual and strong on Arabic; measured better than two alternatives (section 16)
Reranker	BAAI/bge-reranker-v2-m3	int8	Biggest single quality gain in the project; quantised to run in ~2 s on CPU
Keyword search	Own BM25	~110 lines	The algorithm is short, and owning it lets the Arabic tokenizer be shared with the rest

Layer	Choice	Version	Why this one
Vectors	NumPy	2.4.4	An exact dot product over 3,830 vectors is faster than an approximate index at this size
Token counting	tiktoken	0.13.0	Sizing chunks; note its Arabic counts run ~3x the embedder's (section 7)
Answers	google-genai / anthropic	2.25.0 / 0.86.0	Official SDKs for the two supported providers
HTTP API	FastAPI + Uvicorn	0.136.0	Typed request models and streaming responses with very little code
Terminal	Typer + Rich	0.24.1 / 13.9.4	Sub-commands, help text and tables
Tests	pytest	9.1.1	180 test functions, no network or model weights needed
This PDF	ReportLab	5.0.1	Generated from the code snapshot by docs/make_pdf.py

> **What is not used, deliberately.** No vector database (Pinecone, FAISS, pgvector): below roughly a million passages, an exact NumPy dot product is faster than an approximate index and perfectly accurate. No LangChain or LlamaIndex: the pipeline is eight explicit steps, and a framework would hide exactly the parts that needed fixing for Arabic. No OCR: these PDFs already contain text, though damaged.

4 Stage 1 - Reading the documents

Every format is funnelled into one shape, so nothing downstream needs to know where the text came from:

```
Document:
    doc_id      "9f12c4..."           stable id derived from the absolute path
    source      "documents/law.pdf"   where it came from
    title       "Labour Law 1996"     what citations will show
    text        "- Article 1 ..."     the whole document as one string
    metadata    {format, pages, content_hash, repairs applied}
    page_spans  [(0, 2480, "1"), (2480, 5090, "2"), ...]
                  start  end   page   <- maps any character back to a page
```

Format	How it is read	Notes
PDF	pypdf, plain and layout modes	Page boundaries kept as character offsets, so a citation can name a page
.docx	zipfile + ElementTree over word/document.xml	Paragraphs, tables (one row per line) and footnotes, in document order
txt, md, log, source code	Read as UTF-8	Whitespace normalised
csv, tsv	csv.DictReader	One record per block, capped at 2,000 rows
json, jsonl	json module	Pretty-printed, or one record per line
html	Tag stripping	Scripts and styles dropped, entities decoded

Three things happen around the reading itself

- **Extraction is cached.** Layout-mode PDF extraction is the slowest part of ingestion. Its result is stored under the index, keyed by a SHA-256 hash of the file's bytes, so re-indexing after a settings change skips it entirely. The cache currently holds 12 entries. Change one byte of a file and the hash changes, forcing a fresh read.
- **Identical files are indexed once.** Two of the supplied PDFs were the same file under different names. The second is detected by content hash, reported and skipped - otherwise every answer would quote it

twice and spend two of its ten passages on one document.

- **Deleted files are removed.** Ingesting a folder first drops index entries whose file no longer exists there, so a deleted law stops appearing in answers, and a renamed file is re-indexed under its new name instead of being mistaken for a duplicate of its own former name.

Where citation titles come from

Eleven of the twelve PDFs carry the same embedded title - the name of the database they were exported from - which would make every citation identical. The loader therefore looks for the law's own naming clause, the standard formula in Article 1 of Arab legislation:

يسمى هذا القانون (قانون العمل لسنة 1996) ويعمل به the title becomes **Labour Law of 1996**

With no such clause the filename is used, which is why renaming a file changes how it is cited.

5 Stage 2 - Repairing Arabic text

This stage exists because the supplied PDFs do not extract cleanly. Eleven of the twelve came from a legal database that writes Arabic into the PDF as positioned glyphs in display order. Nothing downstream survives that: the keyword index would hold nonsense words, the embeddings would describe nonsense, and any quote in an answer would be unreadable.

Defect	What extraction produced	After repair
Words glued together	اليومالاولتعريفبمقدم	اليوم الاول تعريف بمقدم
Word order reversed on every line	.للضرر تعرض شخص أي :المتضرر	المتضرر: أي شخص تعرض للضرر.
Glyph codes, and a font bug on the tanween ending	وفقغ لاحكام	وفقا لاحكام
Bold words drawn twice and fused	العقارالعقار	العقار

Each defect is detected rather than assumed, so a well-formed PDF is never touched:

- **Glued words** are found by average word length - Arabic words average four to six letters, and a page averaging above nine is run-together text.
- **Reversed word order** is found by punctuation: normal text ends lines with a full stop, reversed text starts them with one. Whole lines are counted rather than tokens, because reversed clause numbering ("1. x" arriving as "x. 1") would otherwise look normal.
- **Glyph codes** (the Unicode presentation forms, U+FB50-U+FEFF) are folded back to ordinary letters. One font maps the tanween ending to the wrong code point; it is recognisable because the character it produces cannot legally end an Arabic word.
- **Print footers** - a browser URL, a timestamp and a page counter on every page - are stripped. They had been landing in every passage, diluting both searches.

The two-extraction trick

Plain extraction gets the letters right but loses the spaces and detaches every article heading from its text. Layout extraction keeps the spacing and the structure but decodes some glyphs into longer junk. Neither is usable alone, so the loader takes the order from layout mode and the letters from plain mode, matching the two line by line on shared words. On the Labour Law this dropped the junk rate from 2.06% to 0.10%.

Normalisation for search

Separately from repair, Arabic is normalised before being indexed or searched, so that the several valid spellings of a word become one term. The same function is applied to documents and to questions, so it only has to be consistent:

Written as	Also written as	Indexed as
الإدارة	الادارة / إدارة / ادارة	اداره
للمادة ١٢	المادة 12	ماده 12
مستشفى	مستشفي	مستشفي

Diacritics and tatweel are removed, Arabic-Indic digits become 0-9, and the definite article and fused prepositions are stripped. Two deliberate exceptions: a leading waw is kept, because in legal vocabulary it is as often part of the word (wizara, ministry) as the conjunction "and"; and negations are kept out of the stopword list, because in law they invert the meaning.

6 Stage 3 - Finding the articles

Laws are quoted by article, not by page, so the article is the unit the system works in. Three heading styles cover every document in this corpus:

- `- Article 61 <topic> effective-date 2019-05-16` - legal-database exports
- `Article 7` alone on a line - the Word file and most official PDFs
- `Article 4- first sentence...` - heading and text sharing a line

The hard part is not finding headings but rejecting false ones: legal text is full of sentences like "under Article 70 of this law". The guard is that article numbers run in order, so a candidate is accepted only if it continues the sequence. A mid-sentence reference almost never does.

```
def _accept(number, bis, previous):
    if previous is None or number == 1:    # first article, or an annex restarting
        return True
    if bis and number == previous:         # "Article 42 bis"
        return True
    return previous < number <= previous + MAX_FORWARD_JUMP   # gaps for repealed articles
```

That the rule works is checkable against the real laws, which is the strongest evidence available:

Document	Articles found	Actual length of the law
Constitution (Word file)	131	131
Civil Code	1,449	1,449
Penal Code	479	476 plus bis articles
Code of Criminal Procedure	371	370
Companies Law	291	289 plus bis articles
Labour Law	142	142

> **Why this mattered so much.** In the database exports, plain extraction put every article number on a page into one block and all the article texts after them. Numbers and texts were both present but no longer connected, so any answer citing an article number would have been guessing. Rebuilding the pairing from layout-mode extraction is what makes a citation such as "Article 170 of the Penal Code" trustworthy.

7 Stage 4 - Chunking and labelling

A passage is one article. Articles longer than the size limit are split into overlapping windows inside their own boundaries, so a chunk never straddles two articles. Of the 3,830 passages in the index, 3,818 carry an article label; the rest are title pages and preambles.

```
Chunk:
    text        "- Article 61 ... fourteen days of annual leave ..."   the article itself
    header      "Labour Law 1996 - Article 61"    prepended for indexing and for the model
    char_start  18423   char_end 19877            offsets back into the document
    pages       ["22"]                           for the citation
    metadata    {article: "61", article_number: 61, topic: ..., format: "pdf"}
    token_count 312

    indexed_text == header + newline + text    <- what is embedded and keyword-indexed
```

Three details carry a lot of weight

- **The label is part of what is searched.** An article rarely names its own law - it says "a worker is entitled to...", not "under the Labour Law a worker...". Prepending "Labour Law 1996 - Article 61" lets a question that names the law find it, and lets the model see which article it is reading.
- **Offsets are kept.** Every chunk knows its character range in the source document. That is what lets a citation name a page, and what lets any quote be checked against the original file.
- **Articles are not packed together.** Grouping short articles into bigger chunks was tried and measured worse at every context size (section 16), so each article stands alone.

> **A trap worth knowing about.** Chunk sizes are measured with tiktoken, an English-trained tokenizer that splits Arabic into roughly three times as many tokens as the embedding model does. The default of 512 therefore gave Arabic chunks a third of the text an English chunk gets, often cutting a legal article in half. This corpus is indexed at 1,200, which restores comparable sizes and still fits inside the embedder's 512-token input window.

8 Stage 5 - Embedding

An embedding is a list of numbers describing what a piece of text means. Texts with similar meanings get similar lists, so "annual leave" lands near "fourteen days of paid leave per year" even though they share no words. Here every passage becomes 1,024 numbers, produced by BAAI/bge-m3 running locally on the CPU.

- **Pooling.** The model emits one vector per token, and they must be reduced to one vector per passage. BGE models are trained to be read from their first token, most others from the average of all tokens. Choosing wrong does not fail - it quietly retrieves worse - so the choice follows the model name.
- **Normalisation.** Every vector is scaled to length 1, which turns comparing two vectors into a single multiplication (a dot product).
- **Asymmetric prefixes.** Some models (the e5 family) are trained with "query:" and "passage:" prefixes and retrieve worse without them; bge-m3 is not, so prefixes are applied only when the model name calls for them.
- **Length-sorted batching.** A batch is padded to its longest member, and this corpus mixes one-line articles with full pages. Sorting by length before batching cut indexing from 211 s to 135 s on the smaller model.

Indexing all 3,830 passages with bge-m3 took about 14 minutes on this machine's CPU - a one-time cost, since re-ingesting only processes new or changed files. Embedding a question takes about 59 ms.

9 The index on disk

The index is three files in a folder. There is no database process, nothing to install and nothing to administer; the whole thing can be copied, backed up, inspected in a text editor, or deleted.

File	Size	What is in it
chunks.jsonl	5.0 MB	One JSON object per passage per line: text, header, offsets, pages, metadata. Line N corresponds to row N of the vector matrix.
embeddings.npy	15.7 MB	A NumPy array of 3,830 x 1024 float32 values: the passage vectors
meta.json	4 KB	Which embedding model and chunk settings built this index, plus a record per source file (content hash, chunk count, repairs applied)
extracted/	12 files	Cached extraction results, keyed by file content hash

Invariants that keep it honest

- **Row N of the matrix is line N of the chunks file.** Everything depends on this. Deleting a source rebuilds both together; adding passages without an embedder deletes the vectors rather than leaving a half-covered matrix.
- **Writes are atomic.** Each file is written under a temporary name and then renamed, so an interrupted save cannot leave a half-written index. If the counts disagree on load, the vectors are dropped and keyword search still works.
- **The index records how it was built.** If the embedding model or the chunk settings change, the mismatch is reported instead of silently producing bad results - vectors from two different models cannot be compared.
- **The matrix is read into RAM, not memory-mapped.** Every query multiplies the whole matrix anyway, so a mapping saves no work; and on Windows an open mapping blocks the next save, which was a real bug here.

10 Stage 6 - Retrieval

Two searches run over the index, because they fail in different places.

Keyword search (BM25)

BM25 scores a passage by how often the question's words appear in it, weighted so that rare words count for more (a passage containing "tanfeeth" is more informative than one containing "the"), with diminishing returns for repetition and a correction for passage length. It is exact: it finds article numbers, names and defined terms that an embedding may blur. It cannot match a paraphrase, and it cannot match across languages at all. Searching all 3,830 passages takes about 0.1 ms.

Meaning search (dense)

The question is embedded with the same model as the passages, and the best matches are the passages whose vectors point most nearly in the same direction - a cosine similarity, which for normalised vectors is one multiplication of the question vector against the whole matrix. It matches paraphrases and crosses languages; it is vaguer about exact identifiers. About 59 ms, dominated by embedding the question.

Fusion

The two rankings cannot be added directly: a cosine similarity of 0.87 and a BM25 score of 24.2 are not on the same scale, and any attempt to weight them needs per-corpus calibration that goes stale. Reciprocal Rank Fusion instead uses only the positions: each list contributes 1/(60 + rank), and a passage that both searches rank highly wins.

```
def reciprocal_rank_fusion(ranked_lists, k=60, weights=None):
    fused = {}
    for list_idx, ranked in enumerate(ranked_lists):
        weight = weights[list_idx] if weights else 1.0
        for rank, (doc_idx, _score) in enumerate(ranked):
            fused[doc_idx] = fused.get(doc_idx, 0.0) + weight / (k + rank + 1)
    return fused
```

Diversification and filtering

- **MMR** (Maximal Marginal Relevance) walks down the fused list and skips passages that are near-copies of ones already chosen, so the ten passages sent to the model are ten different articles rather than one article and nine variations of it. The top hit is always kept.
- **Filters** restrict a search to one file or one format. They are applied before truncation - filtering after cutting to ten would silently return fewer than ten.

11 Stage 7 - Reranking and the context budget

Retrieval compares the question and each passage separately - one embedding per side, or counted words - which is what makes it fast enough to search thousands of passages. It is good at getting the right passage into the top twenty and only approximately right about the order within them.

A cross-encoder reads the question and one passage together and scores how well that passage answers that question. It is far more accurate and far too slow to run over a whole corpus, so it only reorders the twenty candidates that fusion already found. This was the single biggest quality gain in the project:

Setup	Answer ranked first	Answer within 1,000 tokens	Time per question
No reranking	47%	59%	0.2 s
Small model (118M), int8	63%	82%	0.3 s
Large model (568M), float32	73%	82%	5.2 s
Large model, int8 (default)	**76%**	**84%**	**2.2 s**

Two optimisations made the large model usable on a CPU: int8 quantisation (storing the model's weights as 8-bit integers instead of 32-bit floats) and sorting the batch by passage length so that less padding is computed. Together they took it from 11.4 s to 2.2 s per question with no loss of quality - the quantised model measured marginally better, which is within noise.

The context budget

Passages are then taken in order until about 4,000 tokens are filled - roughly ten articles - rather than a fixed count. Article lengths vary more than tenfold here, so "ten passages" can mean 800 tokens or 9,000. Budgeting by tokens makes the cost and the latency of the next step predictable, and section 16 shows it also retrieves better.

12 Stage 8 - Generating the answer

Only this stage leaves the machine. The retrieved passages are numbered and labelled with their law and article, and the instructions are explicit about grounding:

```
system:   You answer questions using only the excerpts provided in <context>.
          - Ground every factual claim in the excerpts...
          - Cite with bracketed numbers matching the excerpt ids, e.g. [2]
          - When an excerpt's location names a law and an article, name them...
          - If the excerpts do not contain the answer, say so plainly...
          - Answer in the language the question was asked in.

          <context>
          <excerpt id="1" source="documents/qistas.pdf"
                   location="Labour Law 1996 - Article 61 (p.22)">
          - Article 61 ... fourteen days of annual leave ...
          </excerpt>
          ... nine more ...
          </context>

user:     How many days of annual leave does a worker get?
```

What the instructions buy

- **Citations that resolve.** The `[n]` markers are parsed back out of the answer afterwards, so the caller gets a structured list of which passages were actually used - not prose that happens to contain brackets. On the web page they are clickable.
- **Refusal as a feature.** The model is told to say the passages do not contain the answer rather than fill the gap. A fluent guess is worse than no answer, because it is indistinguishable from a real one.
- **Answer in the question's language.** Ask in Arabic, get Arabic; ask in English about an Arabic law, get English.

One interface, two providers

Both providers sit behind one small interface: a list of system parts and a list of messages in, text or a stream of text out. Everything provider-shaped stays inside its backend, so the prompt, the citation parsing and the refusal handling are written once.

Concern	Claude	Gemini
Assistant turns are called	assistant	model
Thinking	adaptive, with an effort level	always on, set by level, cannot be disabled
Prompt caching	explicit breakpoint on the context	automatic for repeated prefixes
A refusal arrives as	stop_reason = refusal	a finish_reason on the candidate
Default model	claude-opus-5	gemini-3.8-flash

Transient failures (503 overloaded, 429 rate limited) are retried with a doubling backoff; a 4xx is not retried, because it will not improve. A stream that breaks after text has already arrived cannot be retried either - the partial answer is kept, marked "cut short", and the question can be asked again.

13 How data moves: every hop

The question "where does the data actually go?" has a short answer - almost nowhere - and a precise one. Here is every movement of data in the system:

#	What moves	From	To	Form	Size (this corpus)
1	Document bytes	documents/ on disk	Python process memory	PDF / DOCX binary	70 KB - 2.5 MB each
2	Extracted text	Process memory	Index folder (cache)	JSON, UTF-8	12 files

#	What moves	From	To	Form	Size (this corpus)
3	Passages	Process memory	chunks.jsonl	One JSON object per line	5.0 MB
4	Passage text	Process memory	Embedding model (same process)	Python strings in batches of 16	3,830 passages
5	Vectors	Model	embeddings.npy	float32 matrix	15.7 MB
6	Model weights	Hugging Face (once)	Local model cache	safetensors	2.2 GB embedder + 2.2 GB reranker
7	Question	Terminal or browser	Python process	UTF-8 string, or JSON over HTTP	tens of bytes
8	Question vector	Model	Process memory	1024 float32	4 KB
9	Candidate passages	Index in RAM	Reranker (same process)	20 question-passage pairs	~20 x 300 tokens
10	Prompt	Python process	Gemini or Claude over HTTPS	JSON: system instruction + contents	~3,000 tokens (~12 KB)
11	Answer	Provider over HTTPS	Python process	JSON, or a stream of chunks	tens to hundreds of tokens
12	Answer + sources	Python process	Browser	Server-sent events	a few KB

Hops 1 to 5 happen once per document, at ingestion. Hops 7 to 12 happen per question. Only hops 6 and 10/11 cross the network.

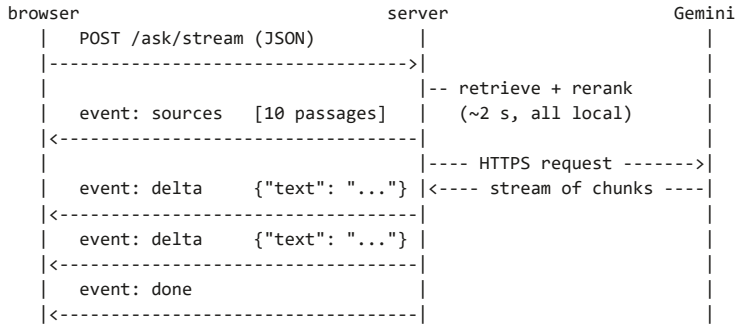
> **What leaves this machine, exactly.** One HTTPS request per question, containing: the system instructions; the ten or so retrieved passages, each labelled with its law, article, page and source filename; and the question itself. A measured example came to 3,003 input tokens, with 26 tokens of answer and 324 tokens of internal reasoning billed back. **Not** sent: the other 3,820 passages, the index, the embeddings, the documents themselves, or anything about the machine. The API key travels in the request header over TLS. If a question matches nothing, no request is made at all.

The one network dependency at startup

The embedding and reranking models are downloaded once from Hugging Face (about 4.4 GB together) and cached in the user profile. After that, indexing and searching work with no network at all - which is why search keeps working with no API key and no internet, and only answering needs either.

Inside the web page

The browser posts JSON and reads a Server-Sent Events stream. Sources arrive first, so the passages can be shown while the answer is still being written:



Server-sent events normally ride on a GET request through the browser's EventSource API, which cannot carry a JSON body. The page therefore posts and parses the event stream itself - about fifteen lines of JavaScript - which keeps the request shape the same as the JSON API.

14 Interfaces: command line, HTTP, web page

Three ways in, one pipeline behind them. All three call the same facade (`RAGPipeline`), so they cannot drift apart.

Command line

Command	What it does
python -m rag ingest documents	Index a folder. Unchanged files are skipped, edited ones re-indexed, deleted ones removed.
python -m rag search "..."	Retrieve passages only. Needs no API key. Shows which retriever found each one, with scores.
python -m rag ask "..."	Retrieve, then answer with citations. Streams as it writes.
python -m rag chat	Multi-turn session; loads the models once, so each question costs only its own time.
python -m rag stats	What is indexed, which provider will answer, which keys were found and where they came from.
python -m rag sources / remove / reset	List indexed files; drop one; delete the index.
python -m rag serve --port 8000	HTTP API and the test page.

A one-off command spends about 16 s loading the two models before it does anything; `chat` and `serve` load them once and then answer in about 2 s plus the provider's time.

HTTP API

Endpoint	Body / result
GET /	The test page
GET /health, GET /stats	Index size, provider, model, which keys were found
GET /examples	A few questions from the evaluation set, for the page
POST /search	{"query": "...", "top_k": 5, "source": null, "rerank": null} -> ranked passages with scores
POST /ask	{"question": "...", "history": []} -> answer, citations, sources, usage
POST /ask/stream	Same body; a sources event, then delta events, then done
POST /ingest	Disabled unless RAG_ALLOW_INGEST=1, because it reads server-side paths

The test page

A single self-contained HTML file with no CDN and no build step, served at the root. It has a question box, an Ask button that streams the answer with clickable citations, and a Search button that retrieves without using an API key. Each passage shows its law and article, its dense, keyword and rerank scores, and the timing. It exists mostly because Arabic renders right-to-left correctly in a browser and cannot in a Windows terminal.

15 Configuration

Everything is one dataclass with one environment variable per field. Values are resolved in this order, each overriding the one before it:

- The defaults in `rag/config.py`, chosen from the measurements in the next section
- A `.env` file in the project folder - where the API key goes
- A real environment variable, which wins over the file, so one command can be run differently without editing anything
- A `Config` object passed in code, or a command-line flag such as `--provider`

Setting	Default	Effect
GEMINI_API_KEY / ANTHROPIC_API_KEY	-	Enables answering. Nothing else needs a key.
RAG_LLM_PROVIDER	auto	auto picks whichever key exists, preferring Gemini
RAG_MODEL	per provider	gemini-3.8-flash or claude-opus-5 unless set
RAG_INDEX_DIR	.rag_index	Where the index lives; several indexes can coexist
RAG_EMBED_MODEL	BAAI/bge-m3	Changing it invalidates the index (re-ingest with --force)
RAG_CHUNK_TOKENS	512	1,200 for this Arabic corpus; remembered by the index
RAG_ARTICLE_CHUNKS	1	One passage per article for legislation
RAG_RERANK / RAG_RERANKER_MODEL	1 / bge-reranker-v2-m3	The cross-encoder stage
RAG_RERANKER_INT8	1	int8 weights on CPU: 5x faster, same quality
RAG_TOP_K / RAG_CONTEXT_TOKENS	12 / 4000	Passages considered, and the budget sent
RAG_DENSE_WEIGHT / RAG_LEXICAL_WEIGHT	1.0 / 1.0	Fusion balance; measure before changing
RAG_EFFORT	medium	Maps onto Gemini's thinking level
RAG_RETRY_ATTEMPTS / RAG_RETRY_SECONDS	2 / 1.5	Retries for 503 and 429, doubling
RAG_DENSE	1	0 falls back to keyword-only, with no model downloads at all

Some settings are remembered by the index itself: chunk size is recorded when the index is built and reused automatically afterwards, so adding a file later cannot silently chunk it differently from the rest.

16 How the system was measured

Every change above was kept or discarded on evidence from a fixed test set: 49 questions over these laws - 41 Arabic, 5 English, 3 article-number lookups - each tied to the article that answers it. A retrieval counts as correct only if the passage comes from the right law and contains the answering text, compared after the same Arabic normalisation the search uses.

Change	Answer ranked first	Answer within 1,000 tokens
Starting point: 1,200-token page windows, e5-small embeddings	43%	43%

Change	Answer ranked first	Answer within 1,000 tokens
One passage per article, labelled with law and article	47%	61%
+ local cross-encoder reranker (int8)	76%	84%
+ bge-m3 embeddings (current)	**80%**	**90%**

With the shipped settings: 94% of questions have the right article in the top five, mean reciprocal rank 0.86, all five English questions and all three article lookups ranked first.

The measurement mistake worth repeating

Switching to one passage per article first looked like a regression: "recall@5" fell from 80% to 65%. The metric was wrong, not the change. A 1,200-token page window holds five or ten articles, so it counts as a hit whenever any of them answers the question, while an article passage has to be the exact article. Comparing five against five handed the page windows four times the text. Measured by what actually matters - how often the answer reaches the model for the same number of tokens - articles win everywhere:

Context sent to the model	Page windows	One article per passage
1,000 tokens	43%	61%
2,000 tokens	57%	71%
4,000 tokens	69%	84%

Things that were tried and rejected

- **Packing short articles together** (400- and 800-token groups): worse at every budget than one article per passage.
- **multilingual-e5-large embeddings:** 71% ranked first after reranking, below both bge-m3 (80%) and even the much smaller e5-small (76%). Bigger is not automatically better.
- **The reranker at full precision:** same quality as int8, 2.3 times slower.
- **Rewriting BM25 for speed:** profiling showed search takes 0.1 ms and the 600 ms figure came from the measurement harness rebuilding the index on every query. The fix was to the harness; the code was left alone.

> **Honest caveat.** The test set was written by the same person who built the system, from the documents being indexed. That flatters the absolute numbers. The comparisons between versions are the trustworthy part, because every version faced identical questions.

17 Testing

180 test functions (7 files) run in about two seconds, with no network, no API key and no model weights.

Test file	Tests	What it covers
conftest.py	-	Shared settings: no model weights, no network
test_arabic.py	27	Spelling normalisation, diacritics, digits, the PDF repairs, the detectors that decide when to apply them
test_legal.py	15	Article heading styles, the ordering guard against false headings, article-aligned chunking

Test file	Tests	What it covers
test_llm.py	36	Provider selection, Gemini and Claude request shapes, refusals, retries, error messages, .env loading
test_rag.py	45	Chunking offsets and overlap, BM25, fusion, MMR, the index round-trip, filters, citation parsing
test_server.py	13	Every HTTP endpoint, the SSE event order, the page, the disabled ingest route
test_sources.py	44	Word files, duplicates, deleted and renamed files, the extraction cache, the context budget

How that is possible

Tests use a deterministic fake embedder: the same text always produces the same vector, derived from a hash. That exercises all the plumbing - persistence, fusion, filtering, offset integrity, citation parsing - without loading 2.2 GB of weights. Embedding quality is a different question, and that is what the evaluation harness in section 16 is for.

The Gemini tests are built the same way: requests are constructed with the real SDK types and handed to a fake transport, so a wrong field name or an invalid value fails in the test suite rather than on the first real call.

What the tests do not cover

- The quality of what the model writes. The tests check that a refusal is handled, that citations are parsed and that the request is well formed; whether the answer is good is judged by reading it.
- Real provider behaviour. A live 503 was observed once in practice and is now retried, but the retry itself is tested against a simulated failure.
- Extraction quality on PDFs other than these twelve. The repairs are guarded by detectors, so an unfamiliar file should pass through untouched rather than be damaged - but only these files have been read closely.

18 Limitations and next steps

Known limitations

- **Scanned PDFs yield nothing.** Extraction reads embedded text only. A scanned page needs OCR first.
- **Some font junk survives.** In the database exports a few glyphs decode to runs of random letters - between 0.1% and 3% of words depending on the file. The word stem before the junk is intact, so retrieval mostly survives, but a quoted sentence can show it.
- **Three test questions still miss.** They ask in everyday words for things the law names differently, or target one entry inside a very long definitions article.
- **Keyword stemming is prefix-only.** Plural and suffixed forms do not match by keyword; the embeddings cover most of that gap.
- **One writer at a time.** Two processes indexing the same folder concurrently will have one overwrite the other.
- **Providers have bad minutes.** Transient failures are retried, but a stream that breaks mid-answer cannot be.

Where it would go next

- **Expand the test set with real questions.** The current one was written by the system's author; questions from actual users would measure what matters.

- **Arabic suffix stemming,** measured rather than assumed - it may help keyword search or may blur distinct terms.
- **Cross-references between laws.** Articles cite other articles constantly; following those links when assembling context would answer layered questions.
- **A GPU, if the corpus grows.** Indexing is the only slow part, and it is the part a GPU would change by an order of magnitude.

Appendix A File-by-file map

File	Lines
rag/__init__.py	38
rag/__main__.py	4
rag/arabic.py	273
rag/bm25.py	130
rag/chunking.py	356
rag/cli.py	388
rag/config.py	207
rag/crossencoder.py	100
rag/embeddings.py	170
rag/generate.py	164
rag/legal.py	146
rag/llm.py	396
rag/loaders.py	578
rag/pipeline.py	269
rag/rerank.py	80
rag/retriever.py	267
rag/server.py	216
rag/store.py	367
rag/web/index.html	342
eval/evaluate.py	347
Total (excluding tests)	**4,838**

The indexed corpus

Titles below are the ones the system extracted from each law's own naming clause; they are what appears in citations.

Document	Articles indexed	Word order repaired
القانون المدني لسنة 1976	1450	yes
قانون العقوبات لسنة 1960	489	yes
قانون التجارة	409	-
قانون اصول المحاكمات الجزائية لسنة 1961	379	yes
قانون الشركات لسنة 1997	334	yes
قسطاس (2)	240	yes
قانون العمل لسنة 1996	148	yes
الدستور-الأردني-مع-كامل-التعديلات-حتى-2022	133	-
قانون التنفيذ لسنة 2007	124	yes

Document	Articles indexed	Word order repaired
قانون البيانات لسنة 1952	75	yes
قانون تشكيل المحاكم النظامية لسنة 2001	27	yes
نظام التأمين الو للمركبات لسنة 2010	22	yes

Appendix B Glossary

Term	Meaning
RAG	Retrieval-augmented generation: find the relevant text first, then ask a model to answer using only that text.
Chunk / passage	A piece of a document indexed as one unit. Here, one article.
Token	The unit models count in - roughly a short word or word-part. Arabic costs more tokens per word than English in most tokenizers.
Embedding	A list of numbers representing meaning, so that similar texts have similar lists. 1,024 numbers per passage here.
Cosine similarity	How closely two embeddings point in the same direction; 1.0 is identical, 0 unrelated.
BM25	The standard keyword ranking formula: rare words count more, repetition has diminishing returns, long passages are penalised.
RRF	Reciprocal Rank Fusion: merge two rankings using positions rather than scores, which avoids comparing incomparable numbers.
MMR	Maximal Marginal Relevance: pick results that are relevant but not near-copies of each other.
Cross-encoder	A model that reads question and passage together and scores relevance - slower and far more accurate than comparing embeddings.
int8 quantisation	Storing model weights as 8-bit integers instead of 32-bit floats: several times faster on a CPU, with little quality cost.
Presentation forms	Unicode code points for the contextual shapes of Arabic letters. Some PDFs store these instead of normal letters.
Tanween	The Arabic grammatical ending written as a doubled vowel mark; one font in this corpus decodes it to the wrong character.
SSE	Server-sent events: a one-way stream of text events over HTTP, used to show the answer as it is written.